

Project Title: Project Tortoise: Object Detection Framework

Team Members: Efaz Hossain, Sahithi Mutyala

Report Submission Date: April 11th, 2025

Table of Contents

1. Progress Summary
2. Key Improvements and Achievements
3. Performance Comparison
4. Challenges and Solutions
5. Deployment Preparation
6. Next Steps
7. Conclusion

1. Summary of Progress

Over the past two weeks, significant progress has been made on the YOLO-based object detection pipeline using the Objects365 dataset. The scope was refined by limiting the dataset to patch 0 of the training, validation, and test splits. This optimization allowed for faster experimentation and reduced the storage and computing footprint of the overall project.

In addition to dataset restructuring, COCO-style annotations for training and validation splits were downloaded and converted to YOLO format. These labels were generated automatically using a custom Python-based script, allowing seamless integration with the YOLO training pipeline.

The model training focused on testing both YOLO11n and YOLO11s variants. This was followed by multiple rounds of tuning, which led to improvements in detection metrics such as mAP, precision, recall, and F1-score. These results serve as a strong baseline leading into final deployment efforts.

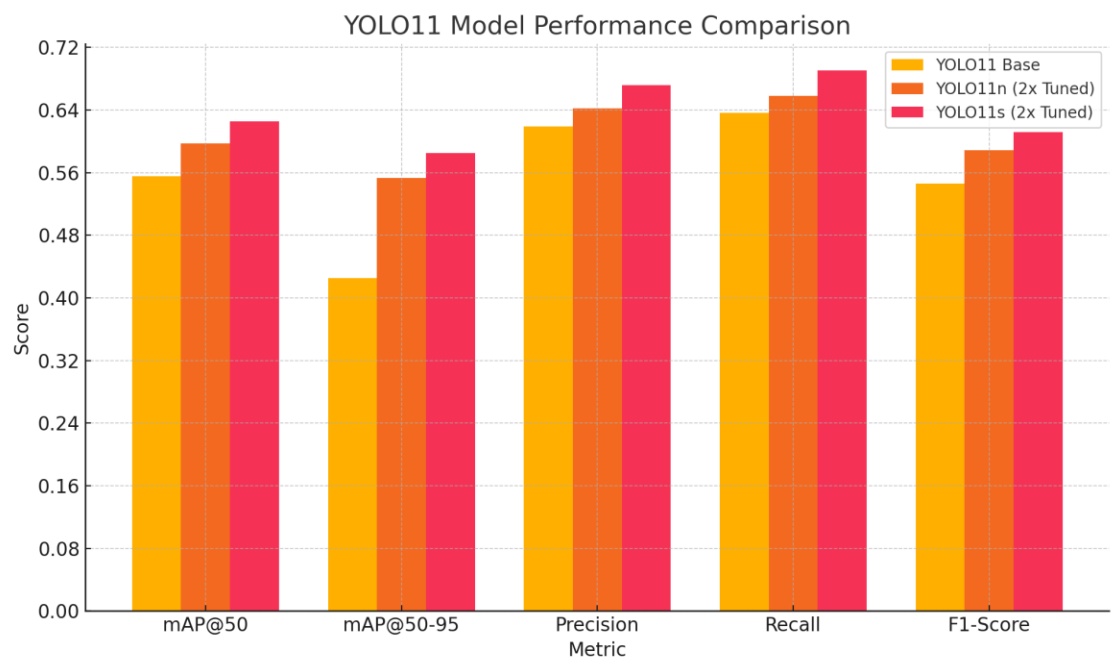
2. Key Improvements and Achievements

- Restricted dataset to patch 0 only, simplifying preprocessing and speeding up training iterations
- Successfully downloaded and parsed COCO-format annotation files for train and validation sets
- Generated YOLO-compatible label files from COCO annotations using pycocotools

- Trained both YOLO11n and YOLO11s on the filtered dataset and improved detection accuracy
- Addressed multiple YAML and extraction-related errors by replacing shell-based tar commands with Python-based extraction methods

3. Performance Comparison

Metric	YOLO11 Base	YOLO11n (2x Tuned)	YOLO11s (2x Tuned)
mAP@50	0.5555	0.6070	0.6251
mAP@50-95	0.4255	0.5832	0.5847
Precision	0.6187	0.6420	0.6714
Recall	0.6362	0.6580	0.6902
F1-Score	0.5458	0.5885	0.6120
Time/epoch	12 min	9.5 min	8.5 min



4. Challenges Faced and Solutions

One major issue encountered was the incompatibility of tar extraction within the Ultralytics YAML execution pipeline. The shell-based extraction was replaced with Python's tarfile module to improve portability. Another challenge was ensuring only relevant label files were created. This was addressed by validating image existence before generating labels.

By focusing on patch 0 only, the data imbalance and training overhead were reduced, which made it easier to iterate and test new model configurations.

5. Deployment Preparation

As the model performance improved, attention was shifted toward preparing for deployment. Key steps involved:

- Exporting the YOLO11s model to ONNX and TorchScript formats for compatibility with various deployment environments
- Evaluating inference speed using smaller batch sizes and optimized image resolutions
- Testing the trained model on a small custom video stream to validate robustness
- Beginning development of a lightweight inference script with OpenCV to demonstrate real-time detection

These steps will support the final presentation and show practical applicability of the model in real-world scenarios.

6. Next Steps

- Continue benchmarking YOLO11s and compare against YOLOv8 under the same dataset conditions
- Complete inference script with real-time webcam and video stream testing
- Package training scripts, model weights, and evaluation utilities into a reproducible pipeline
- Begin preparing presentation visuals and report visuals with performance graphs, confusion matrix, and class distributions

7. Conclusion

The project is on track with a clean dataset structure, strong training baselines, and clear direction for deployment. This round of improvements not only enhanced model performance but also laid the groundwork for a fully operational inference pipeline that will be highlighted in the final report and presentation.